

Programació modular

Índex

Programació modular.....	1
1. Definició.....	1
2. Principals avantatges.....	2
2.1 Reutilització de codi.....	2
2.2 Escalabilitat de l'aplicació.....	2
2.3 Portabilitat del codi.....	2
3. Codi espagueti.....	2
4. Descomposició.....	3
5. Mètodes estàtics i no estàtics (d'instància).....	4
5.1 Mètodes no estàtics (d'instància).....	4
5.2 Mètodes estàtics.....	5
6. Accessibilitat de les variables.....	6
7. Modificadors d'accés.....	7
8. Inner Class.....	8

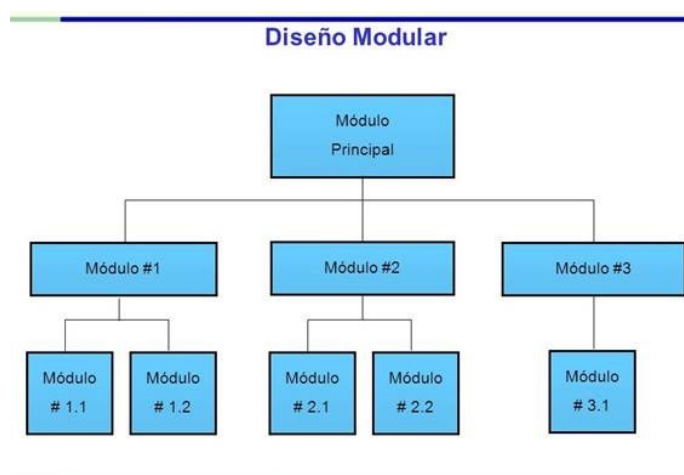
1. Definició

Partim de la idea de "**divideix i guanyaràs**", és a dir, sempre serà més fàcil resoldre un problema si aquest està dividit en parts més petites i més fàcils d'entendre o manipular.

La wikipèdia ens diu el següent sobre la programació modular:

*"La **programació modular** és una tècnica de disseny del programari que emfasitza separar la funcionalitat d'un programa a mòduls independents, intercanviables, tal que cadascun d'ells conté tot el necessari per a executar només un aspecte de la funcionalitat desitjada."*

Consisteix en descompondre un problema complex o gran en parts més petites. Aquestes parts reben el nom de funcions, mètodes, procediments, subprogrames, etc. Normalment en Java els anomenarem **mètodes**.



2. Principals avantatges

- Reduir la complexitat del problema
- Reduir la mida del problema
- Afavoreix l'enteniment del problema
- Facilita la lectura del codi
- Ajuda a ser més clara la lògica del programa
- Facilita la cooperació entre programadors
- **Reutilitza codi.**
- Permet l'**escalabilitat de l'aplicació**
- Permet la **portabilitat del codi**
- Protegeix contra efectes col·laterals (destrucció accidental de dades de programa)
- Permet plantejar una solució completa del problema, i més tard profunditzar en els detalls.
- La depuració és més fàcil de realitzar ja que primer se corregeixen errors en els mòduls de nivell inferior.

2.1 Reutilització de codi

Es parla de **codi reutilitzable** quan una aplicació incorpora parts de codi (estructures, principalment) que es repeteixen i poden ser codificades només una vegada, però s'utilitzen més d'una vegada.

2.2 Escalabilitat de l'aplicació

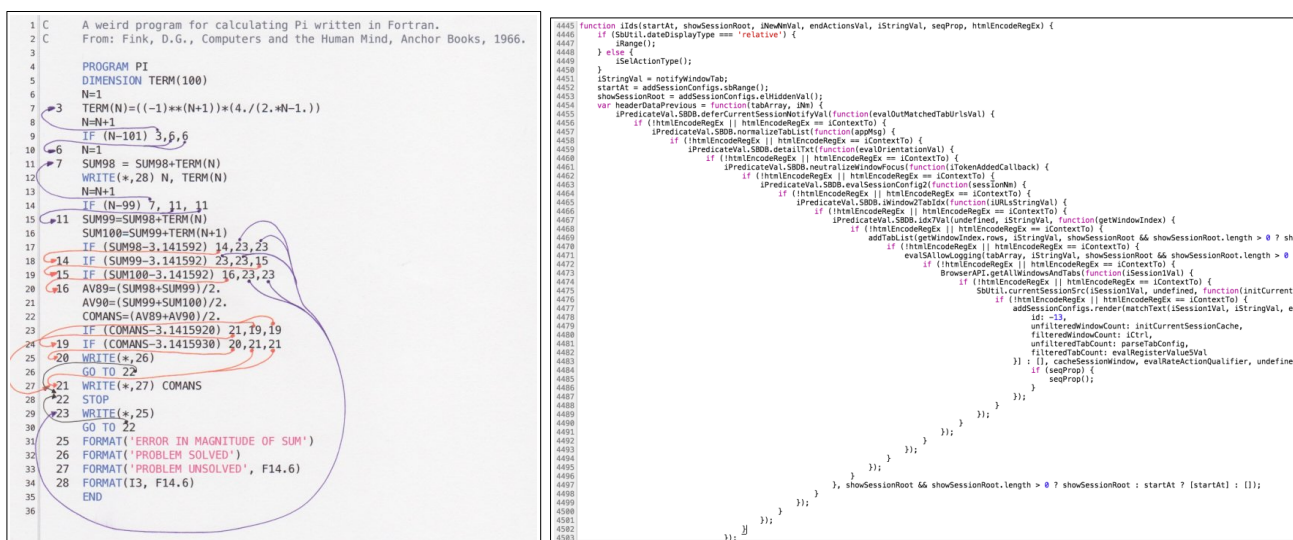
Entenem per **escalabilitat d'una aplicació** la facilitat amb la qual aquesta es pot ampliar i adaptar als canvis en les seves especificacions i funcionalitats.

2.3 Portabilitat del codi

Podem definir la **portabilitat del codi** com la facilitat d'utilitzar un mateix programa en plataformes diferents (maquinari i programari), minimitzant l'esforç d'adaptar-ho.

3. Codi espagueti

Al principi de la programació va aparèixer les sentències GOTO, sentències que permetien saltar d'un punt a un altre en el codi, aquest tipus de programes, se'ls coneixia amb el nom despectiu de **codi espagueti** perquè era impossible seguir fàcilment el flux del programa a causa del nombre de salts que realitzava durant l'execució:



Posteriorment van aparèixer els programes estructurats que incorporaven estructures de control de flux i feia més fàcil el seu seguiment i control, però avui en dia, si els programes són llargs i extensos, és necessari utilitzar la programació encara més dividida en subprogrames = programació modular.

Hi ha diferents mètodes de com es pot descompondre un programa en subprogrames:

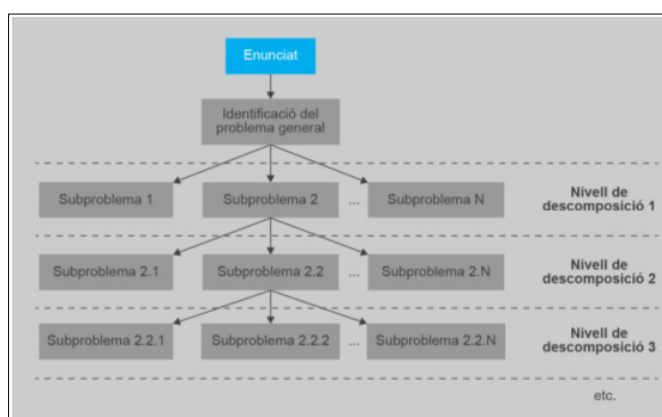
El **disseny descendent** (*disseny top-down*) és la tècnica de programació modular més utilitzada i consisteix en la descomposició del problema general en problemes més simples, denominats subproblemes, els més complexos dels quals es tornen a descompondre en altres subproblemes. S'anomena descendent perquè partint del problema gran passa a problemes més petits als quals donarà solució.

El **disseny ascendent** (*disseny bottom-up*) és una altra manera de resoldre el problema. En aquest cas es comença plantejant problemes bàsics per als quals se cerca la corresponent solució i, a continuació, es construeix totalment el problema.

4. Descomposició

Un mètode de programació preserva la **descomposició** modular si facilita que es divideixi un problema de programació en un nombre petit de subproblemes menys complexos, interconnectats per estructures senzilles i prou independents per poder ser tractats individualment.

Si un subproblema encara presenta un cert nivell de complexitat, es pot continuar descomponent en un conjunt de subproblemes nous. Aquest és un dels conceptes principals dins dels mètodes de programació, i és conegut com a disseny descendent.



5. Mètodes estàtics i no estàtics (d'instància)

Els mètodes d'una classe poden ser de dos tipus: estàtics i no estàtics (o també anomenats d'instància).

5.1 Mètodes no estàtics (d'instància)

Els mètodes no estàtics (o mètodes d'instància) són aquells que depenen d'un objecte i que per tant s'invoquen a partir de la instanciació d'un objecte.

Posem un exemple, imaginem aquests mètode de la classe **Converter**, que converteixen de metres a km (**m2km**) i de km a milles (**km2Milles**):

```
public class Converter {
    public double m2Km(double m) {
        return m / 1000;
    }
    public double km2Milles(double km) {
        return km * 0.621371;
    }
}
```

Ara, imaginem que tenim un altre mètode dins de la classe que vol utilitzar aquests mètodes, és a dir, per convertir de metres a milles (**m2Milles**), podem utilitzar el mètode de metres a kilòmetres (**m2Km**) i de kilòmetres a milles (**km2Milles**):

Tenim la possibilitat de cridar els mètodes amb la paraula reservada **this** o sense **this**.

```
public double m2Milles(double m) {
    return this.km2Milles(this.m2Km(m));
}
```

```
public double m2Milles(double m) {
    dreturn km2Milles(m2Km(m));
}
```

Per utilitzar els mètodes es requereix un mètode **main**, aquest pot situar-se directament a la classe **Converter** o crear una altra classe, per exemple **ConverterTest** i situar el mètode **main**, en els dos casos haurem d'instanciar la classe **Converter** per poder cridar el mètodes:

Situem el mètode main dins la classe Converter	Creem una altra classe ConverterTest amb el mètode main
<pre>public class Converter { public static void main (String[] args) { Converter conv = new Converter(); double value = 215012.0; double kilometers = conv.m2Km(value); System.out.println(kilometers); } }</pre>	<pre>public class ConverterTest { public static void main(String[] args) { Converter conv = new Converter(); double value = 215012.0; double kilometers = conv.m2Km(value); System.out.println(kilometers); } }</pre>

5.2 Mètodes estàtics

Els mètodes estàtics són aquells que **no** depenen d'un objecte i que per tant s'invoquen a partir de la classe.

Posem un exemple, imaginem aquest mètode **sum** de la classe **Arithmetic**:

```
public class Arithmetic {
    public static int sum(int a, int b) {
        return a + b;
    }
}
```

Si es vol fer servir aquest mètode dins de la classe **Arithmetic**, farem servir el mètode cridant-lo directament o utilitzant el nom de la classe al davant:

Sense utilitzar el nom de la classe al davant del mètode:	Utilitzant el nom de la classe al davant del mètode:
<pre>public class Arithmetic { public static int sum(int a, int b) { return a + b; } public static int sum3(int a,int b,int c) { return sum(a, b) + c; } }</pre>	<pre>public class Arithmetic { public static int sum(int a, int b) { return a + b; } public static int sum3(int a,int b,int c) { return Arithmetic.sum(a, b) + c; } }</pre>

De la mateixa manera, si aquesta classe tingués un mètode **main**, el mètode **static** es pot cridar amb el nom al davant o sense:

Sense utilitzar el nom de la classe al davant del mètode:	Utilitzant el nom de la classe al davant del mètode:
<pre>public class Arithmetic { public static int sum(int a, int b) { return a + b; } public static int sum3(int a,int b,int c) { return sum(a, b) + c; } public static void main (String[] args) { int res = sum(2, 4); System.out.println(res); } }</pre>	<pre>public class Arithmetic { public static int sum(int a, int b) { return a + b; } public static int sum3(int a,int b,int c) { return sum(a, b) + c; } public static void main (String[] args) { int res = Arithmetic.sum(2, 4); System.out.println(res); } }</pre>

O des d'una altra classe, en aquest cas és necessari cridar-lo amb el nom de la classe al davant:

```
public class Program {
    public static void main(String[] args) {
        int res = Arithmetic.sum(2, 4);
    }
}
```

6. Accessibilitat de les variables

L'àmbit d'existència d'una variable és la part de la classe en què la variable pot ser referenciada o s'hi pot accedir pel seu nom.

Haurem de definir si les variables utilitzades en el nostre programa seran globals o locals:

Variables globals: Seran accessibles des de qualsevol punt del programa i subprogrames.

Variables locals: Seran accessibles només dins del mòdul on han estat definides.

En Java, quan declarem una variable, aquesta és accessible només dins l'estructura on es declara. En el cas dels arguments, només són visibles en el mètode en el qual s'usen. Posem un exemple:

```
public double m2km(double m) {  
    double km;  
    km = m / 1000;  
    return km;  
}
```

En aquest cas les variables locals "m" i "km" només són accessibles dins del mètode **m2km**.

Un altre exemple:

```
public int sumElements(int[] array) {  
    int sum = 0;  
    for(int i = 0; i < array.length; i++) {  
        int elem = array[i];  
        sum = sum + elem;  
    }  
    return sum;  
}
```

Les variables locals "sum" i "array" només són accessibles dins del mètode sumElements, però la variable local "i" i "elem" només són visibles dins del bucle for.

7. Modificadors d'accés

El modificador d'accés dels mètodes poden ser public, protected, default i private, els seus accessos són els següents:

	Mateixa Classe o niada	Classe en el mateix Package	Subclasses (Classes heretades)	Totes
public	Si	Si	Si	Si
protected	Si	Si	Si	No
default	Si	Si	No	No
private	Si	No	No	No

public: les classes, mètodes o membres són accessibles des de qualsevol lloc del programa en el mateix package o a packages diferents.

protected: els mètodes o membres son accessibles dins del mateix package o subclasses en packages diferents.

default: en el cas de no especificar cap modificador d'accés, els membres, classes o mètodes només són accessibles dins del mateix package. També s'anomena **package-private**.

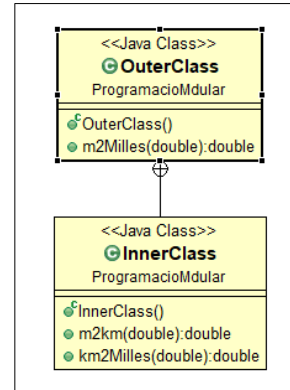
private: els mètodes, membres o classes internes només són accessibles des de la mateixa classe. Les classes o interfícies de primer nivell (no internes) no es poden declarar com privades.



8. Inner Class

Una classe interna o *Inner Class*, és una classe definida dins d'una altra classe, anomenem aquesta última classe com *Outer Class*. És un tipus de classe niada. Les classes internes ens permeten agrupar classes relacionades i controlar la seva visibilitat.

Representació UML de diagrama de classes d'una classe interna:



El modificador d'accés d'una classe interna pot ser **public**, **protected**, **default** o **private**.

Imaginem que realitzem una classe externa que calcula la conversió de metres a milles, però per realitzar aquest càlcul, haurà de fer servir uns mètodes estan definits en una classe interna, si es defineix com private, serà només la classe externa que pot utilitzar-los.

```

package package1;
public class OuterClass {

    public double m2Milles(double m) {
        //TODO
    }

    //Inner Class
    private class InnerClass {

        public double m2km(double m) {
            return m / 1000;
        }
        public double km2Milles(double km) {
            return km * 0.621371;
        }
    }
}
  
```

Podria ser:
public
protected
default
private

Ús de la classe interna dins de la classe externa. La classe interna és una classe d'instància, per tant l'haurem d'instanciar i utilitzar l'objecte per cridar els seus mètodes:

```

package package1;
public class OuterClass {

    public double m2Milles(double m) {
        InnerClass innerClass = new InnerClass();
        return innerClass.m2km(innerClass.km2Milles(m));
    }
    //Inner Class
    ...
}
  
```

Ús de la classe interna fora de la classe externa, per exemple en una altra classe del mateix package, en aquest cas cal declarar el *path* a on tenim la classe interna `OuterClass.InnerClass`, la classe interna haurà de ser pública, default o `protected`, però mai `private`, sinó no podrem fer servir els seus mètodes:

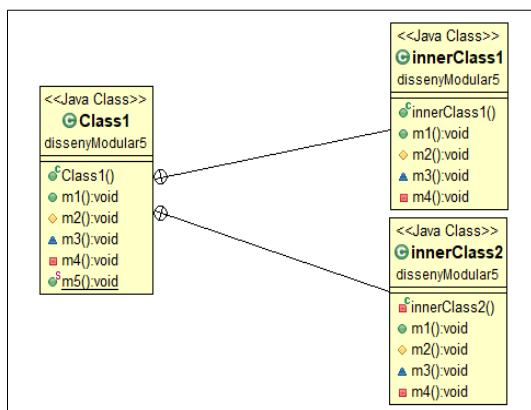
```
package package1;
public class Test {
    public static void main(String[] args) {

        OuterClass myClass = new OuterClass();
        myClass.m2Milles(1000);

        //Use InnerClass
        OuterClass.InnerClass innerClass = myClass.new InnerClass();

        innerClass.km2Milles(1000);
        innerClass.m2km(1000);
    }
}
```

Exemple de representació dels diferents accessos en el diagrama de classe: (no oficial)



●	El cercle verd indica que els mètodes o les classes són publics .
◆	El rombe groc indica que els mètodes o les classes són protected .
▲	El triangle indica que els mètodes o les classes són default .
■	El quadratet vermell indica que els mètodes o les classes internes són privates .
<u>m5()</u>	Els mètodes subratllats indiquen que els mètodes són statics .

Llicència

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is in <http://www.gnu.org/licenses/fdl.txt>